

01 — Environment and Reproducibility

Research objective

Document the computational environment so that every experiment in this study can be reproduced exactly by a reader with the same setup. Reproducibility is not optional — it is a core research principle (see `research_proposal.md` §12.6).

What this notebook does

1. Records Python version, package versions, and OS.
2. Validates that all required packages are importable.
3. Seeds all random-number generators and confirms deterministic behaviour.
4. Checks data paths exist.
5. Writes a reproducibility manifest that downstream notebooks reference.

```
In [2]: import sys, os, platform, pathlib, datetime, json
import hashlib
import numpy as np
import random

# Confirm we are in the right place
PROJ = pathlib.Path(os.environ.get("AIINV_ROOT", ".")).resolve()
print(f"Project root : {PROJ}")
print(f"Python      : {sys.version}")
print(f"Platform     : {platform.platform()}")
print(f"NumPy        : {np.__version__}")
print(f"Working dir   : {pathlib.Path.cwd()}")
```

```
Project root : D:\Brain\07_Research\AI-INVENTORY-OPTIMIZATION\08_notebooks
Python      : 3.12.3 (tags/v3.12.3:f6650f9, Apr  9 2024, 14:05:25) [MSC v.1938 64 b
it (AMD64)]
Platform    : Windows-11-10.0.26200-SP0
NumPy       : 2.5.2
Working dir  : d:\Brain\07_Research\AI-INVENTORY-OPTIMIZATION\08_notebooks
```

Environment snapshot

The table below captures the runtime environment. This snapshot is saved to `05_experiments/manifests/environment.json` and included in every experiment output so results can be traced to a specific software configuration.

```
In [3]: # Collect all required package versions
required = ["pandas", "numpy", "statsmodels", "sklearn", "scipy",
            "matplotlib", "seaborn"]
versions = {}
```

```

for pkg in required:
    try:
        mod = __import__(pkg)
        versions[pkg] = getattr(mod, "__version__", "unknown")
    except ImportError:
        versions[pkg] = "MISSING"

env = {
    "timestamp": datetime.datetime.now().isoformat(),
    "python": sys.version,
    "platform": platform.platform(),
    "packages": versions,
    "project_root": str(PROJ),
}

# Print nicely
for k, v in env.items():
    if k != "packages":
        print(f"{k:20s}: {v}")
print()
for pkg, ver in sorted(versions.items()):
    status = "✅" if ver != "MISSING" else "❌"
    print(f" {status} {pkg:15s} {ver}")

```

```

timestamp           : 2026-08-28T16:33:48.931749
python              : 3.12.3 (tags/v3.12.3:f6650f9, Apr  9 2024, 14:05:25) [MSC v.19
38 64 bit (AMD64)]
platform            : Windows-11-10.0.26200-SP0
project_root        : D:\Brain\07_Research\AI-INVENTORY-OPTIMIZATION\08_notebooks

```

```

✅ matplotlib      3.11.1
✅ numpy           2.5.2
✅ pandas          3.0.5
✅ scipy           1.18.1
✅ seaborn         0.13.2
✅ sklearn         1.9.0
✅ statsmodels     0.14.6

```

Seed control

All random processes (Python `random`, NumPy, and model training when applicable) are seeded to `GLOBAL_SEED`. This ensures that stochastic model initialisation, data shuffling, and any sampling produce identical results across runs.

Important: The LLM notebook (`14_llm_forecasting.ipynb`) additionally sets `temperature=0` and records the model name/version, since LLM determinism is not guaranteed even with seeds.

In [4]: `GLOBAL_SEED = 42`

```

random.seed(GLOBAL_SEED)
np.random.seed(GLOBAL_SEED)
os.environ["PYTHONHASHSEED"] = str(GLOBAL_SEED)

# Verify determinism: same seed → same sequence

```

```

a = np.random.randn(5)
np.random.seed(GLOBAL_SEED)
b = np.random.randn(5)
assert np.array_equal(a, b), "Determinism check FAILED"
print("✅ Determinism check passed: same seed → same sequence")
print(f"    Sample: {a.round(4)}")

```

✅ Determinism check passed: same seed → same sequence
 Sample: [0.4967 -0.1383 0.6477 1.523 -0.2342]

Data path verification

The raw data should exist at:

- `02_data/dataset_01_m5/raw/` — Walmart M5 (sales_train_validation.csv, calendar.csv, sell_prices.csv)
- `02_data/dataset_02_grocery/raw/` — Corporación Favorita (train.csv, stores.csv, items.csv, oil.csv, holidays_events.csv, transactions.csv)

If any path is missing, the corresponding audit/EDA notebooks will document the issue explicitly rather than silently failing.

```

In [5]: # Verify data paths
data_paths = {
    "M5 sales": PROJ / "02_data/dataset_01_m5/raw/sales_train_validation.csv",
    "M5 calendar": PROJ / "02_data/dataset_01_m5/raw/calendar.csv",
    "M5 sell_prices": PROJ / "02_data/dataset_01_m5/raw/sell_prices.csv",
    "Fav train": PROJ / "02_data/dataset_02_grocery/raw/train.csv",
    "Fav stores": PROJ / "02_data/dataset_02_grocery/raw/stores.csv",
    "Fav items": PROJ / "02_data/dataset_02_grocery/raw/items.csv",
    "Fav oil": PROJ / "02_data/dataset_02_grocery/raw/oil.csv",
    "Fav holidays": PROJ / "02_data/dataset_02_grocery/raw/holidays_events.csv",
    "Fav transactions": PROJ / "02_data/dataset_02_grocery/raw/transactions.csv",
}

all_ok = True
for label, p in data_paths.items():
    exists = p.exists()
    status = "✅" if exists else "❌"
    size = f"{p.stat().st_size / 1e6:.1f} MB" if exists else "MISSING"
    print(f" {status} {label:20s} {size:10s} {p.name}")
    if not exists:
        all_ok = False

if all_ok:
    print("\n✅ All data paths verified.")
else:
    print("\n⚠️ Some data paths missing – see 02_data_acquisition_and_audit.ipynb

```

✖	M5 sales	MISSING	sales_train_validation.csv
✖	M5 calendar	MISSING	calendar.csv
✖	M5 sell_prices	MISSING	sell_prices.csv
✖	Fav train	MISSING	train.csv
✖	Fav stores	MISSING	stores.csv
✖	Fav items	MISSING	items.csv
✖	Fav oil	MISSING	oil.csv
✖	Fav holidays	MISSING	holidays_events.csv
✖	Fav transactions	MISSING	transactions.csv

⚠ Some data paths missing – see 02_data_acquisition_and_audit.ipynb for details.

Write reproducibility manifest

This JSON file is the authoritative record of the software environment for this research session. Every experiment output directory should include a copy.

```
In [6]: manifest = env.copy()
manifest["seed"] = GLOBAL_SEED
manifest["data_paths"] = {k: str(v) for k, v in data_paths.items()}
manifest["data_available"] = {k: v.exists() for k, v in data_paths.items()}

out_dir = PROJ / "05_experiments/manifests"
out_dir.mkdir(parents=True, exist_ok=True)
out_path = out_dir / "environment.json"
with open(out_path, "w") as f:
    json.dump(manifest, f, indent=2, default=str)
print(f"✅ Manifest written to {out_path}")
```

✅ Manifest written to D:\Brain\07_Research\AI-INVENTORY-OPTIMIZATION\08_notebooks\05_experiments\manifests\environment.json

Verification checklist

Check	Result
Python 3.12+	✅
pandas ≥ 2.0	See output above
numpy ≥ 1.26	See output above
statsmodels ≥ 0.14	See output above
scikit-learn ≥ 1.3	See output above
scipy ≥ 1.11	See output above
matplotlib ≥ 3.7	See output above
seaborn ≥ 0.12	See output above
Global seed = 42	✅ Determinism confirmed
Data paths exist	See output above

Check	Result
Manifest written	

Decision: Environment is valid for proceeding to data audit and EDA. **Next notebook:**
`02_data_acquisition_and_audit.ipynb`